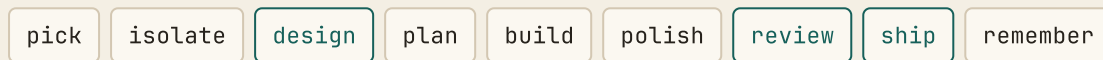


FACILITATOR'S WORKSHOP BOOK

From Issue to Merged PR with a Team of Agents

A hands-on workshop that takes one real issue through an agent-assisted workflow: isolated worktrees, guarded sandboxes, spec-first design, subagent builds, automated polish and a human review at the end.



Format: 2 h 30 min · 6–20 people · talk + live demo + hands-on

Companion: `deck.html` (open in a browser, ← → to navigate, F for fullscreen)

00 How to use this book

This is a run sheet. Each module lists what to say, what to type and what should appear, and what to do if it doesn't. Read it once end to end, then keep it open on a second screen during the session.

The workshop is about **what the workflow lets you do when you pick up an issue**, not about how the dotfiles are wired. Show config only when someone asks. People should leave thinking “I can run three issues at once and only look at them when they need me,” not “I need to learn Nix.”

MARKER	MEANING
SAY	Talking points. Paraphrase, don't read aloud.
DO	Exact commands and prompts for the live demo.
EXPECT	What a successful run looks like.
IF IT BREAKS	Fallback. Agents are nondeterministic, so every live step has one.

Contents

- 01 Workshop at a glance: outcomes, audience, agenda
- 02 Preparation: demo repo, machine, fallbacks, checklists
- 03 Run sheet: nine modules, minute by minute
- 04 Hands-on lab for attendees
- 05 Take-home kit: the workflow without Nix
- 06 Facilitation: pacing, hard questions, FAQ
- 07 Troubleshooting
- 08 Appendix: cheat sheet and presenter file map

01 Workshop at a glance

Outcomes

By the end, attendees can:

1. Run an agent on an issue in its **own worktree**, so it never touches their working copy.
2. Explain why a **sandbox and guard hooks** make it reasonable to turn permission prompts off.
3. Get a **spec and a plan** from the agent before code, and use the plan to resume after a crash.
4. Review agent output in their editor as a **normal branch diff**.
5. Set up at least one of the take-home pieces on their own machine the same day.

Audience

Working developers who have tried an AI coding agent at least once. No Nix, Neovim or Rust knowledge needed: the demo repo is small enough to follow without knowing Rust, and every idea gets a tool-agnostic version in Chapter 5.

Agenda (2 h 30 min)

TIME	BLOCK	DECK SLIDES	MODE
0:00	Opening: why a bare agent gets in its own way	1–3	talk
0:10	M1 Pick: read the issue, recall context	4	demo
0:18	M2 Isolate: worktree, direnv, scoped agents	5–6	demo
0:30	M3 Unattended and safe: sandbox and guard hooks	7	demo
0:42	M4 Parallel: three issues, tab states, statusline	8	demo
0:50	M5 Design: brainstorm, spec, <code>/spec-review</code>	9	demo
1:05	Break (let the build run during it)	–	–
1:15	M6 Plan and build: task graph, subagents, resume	10–11	demo
1:25	M7 Polish: three review rounds, cleanup	12	demo
1:32	M8 Review and ship: diff in editor, <code>/commit</code> , <code>/pr</code>	13–14	demo
1:42	M9 Remember: memory across sessions	15–16	demo
1:50	Hands-on lab (Chapter 4)	17–18	attendees
2:20	Wrap-up, Q&A, feedback	–	talk

TIMING TRICK

The build (M6) is the only slow part. Start it **just before the break** so the agent works while people get coffee. After the break the plan checkboxes show what got done, which itself demonstrates “the plan is the progress log.”

02 Preparation

Most workshop failures happen before anyone arrives. Build the demo repo a week early and run the whole flow once, end to end, on the machine you'll present from.

2.1 The demo repo: `tally`

A tiny Rust CLI that counts lines, words and bytes from a file or stdin. It's small enough to read in a minute and big enough for a real spec. Rust makes direnv link the `rust-engineer` agent automatically, which is itself something to show.

Create it as a public GitHub repo you control with **three seeded issues**. You'll fix #1 live; the other two exist to show parallel work.

```
# one-time setup, a week before
cargo new tally && cd tally
# write src/main.rs: read path arg or stdin, print "lines words bytes".
# Deliberately compute average words per line as words / lines
# so empty input divides by zero and panics.
cat > .envrc <<'EOF'
use flake # or: use nix / layout rust. Anything that puts cargo on PATH
EOF
git add -A && git commit -m "init tally"
gh repo create tally --public --source . --push

gh issue create -t "Panics on empty input" \
  -b "echo -n '' | tally → thread 'main' panicked: attempt to divide by zero"
gh issue create -t "Add --json output" \
  -b "Scripts want machine-readable output: {\"lines\":..,\"words\":..,\"bytes\":..}"
gh issue create -t "Count unicode words correctly" \
  -b "Words split on ASCII whitespace only; NBSP-separated words count as one."
```

RESET BETWEEN REHEARSALS

Tag the seed commit (`git tag seed`). Before every rehearsal and before the real session, delete the old checkout and worktrees and re-clone. Stale `docs/specs` or leftover memory entries give the audience a different demo from the one you rehearsed.

2.2 Machine setup

ITEM	WHY
Terminal font 18–20 pt, light theme	Projectors wash out dark themes. The statusline palette was built for light backgrounds.
tmux with 3 windows, named by issue	Makes the tab-state glyphs (⚡ ? ✓) visible to the room.
Browser tab with the GitHub issue list	Shows the issue before the agent reads it.
Phone with the ntfy app, screen mirrored or in hand	M4: a notification arriving on the phone always gets a reaction.
Notifications off (Slack, mail)	Screen privacy and focus.
Signed in: <code>gh auth status</code> , Claude subscription	An auth prompt in the middle of a demo stalls everything.
Rate limits checked on the statusline	Don't start a live demo near a 5 h limit.

2.3 Record fallbacks

Record each demo module once while rehearsing (`asciinema rec m5-spec.cast` , or a screen recording). If a live step stalls for more than ~60 s, say “this is the recording from yesterday's run” and play it. Nobody minds, and it's better than a silent minute.

Also keep **finished artefacts** in a branch `demo/done` : a completed spec, plan and fixed code. If the build is still running when you need to review, check that branch out.

2.4 Screen hygiene

NEVER ON SCREEN

- The `secrets/` directory, `~/.config/sops` , any `/run/secrets/*` path.
- The ntfy topic (it's effectively a password for your notifications).
- Work GitLab instances, MR titles or anything from the employer's repos.
- Your private `~/.claude/CLAUDE.md` . Read it first if you want to show it.
- Memory search results from real projects. Use a clean namespace for `tally` .

2.5 Checklists

T–7 DAYS

- ☐ Demo repo and three issues created
- ☐ Full end-to-end rehearsal, timed
- ☐ Fallback recordings captured
- ☐ `demo/done` branch pushed

T–30 MIN

- ☐ Fresh clone at `seed` , old worktrees removed
- ☐ tmux windows open and named
- ☐ Deck open, fullscreen tested on the projector
- ☐ Font size checked from the back row

- Take-home kit (Ch. 5) tested on a clean machine

- Statusline shows healthy rate limits

T-1 DAY

- `custom.claude.bashGuard.enable = true` on the presenting host. It's **off by default** and M3 depends on it

- Wi-Fi works; phone hotspot as a fallback

- Water

- `just switch` done, nothing pending

- Second rehearsal from a fresh clone

- Share link to the take-home kit ready

03 Run sheet

Nine modules, each one stage in the life of an issue. Keep the lifecycle strip (deck slide 3) in mind. Every module starts by saying which stage it is.

Opening

0:00 · 10 min · slides 1-3

SAY

“Agents can already write the code. What stops you giving one a real issue and walking away is everything around the code: it edits the checkout you're in, it asks permission every thirty seconds, it forgets everything, and it hands you a diff full of narrating comments.”

“Today we take one issue from the tracker to a merged PR. You'll see the nine stages, and you'll see that I step in at only four of them.”

On slide 2, ask the room: “*Hands up if an agent has ever edited a file you had open.*” It gets people talking early.

M1 · Pick

0:10 · 8 min · slide 4

SAY

The agent reads the issue itself, through the GitHub MCP server. At work the same thing happens against GitLab, read-only. Then it checks memory in case we've been near this code before.

DO

```
cd ~/demo/tally/main
a                                # sandboxed Claude
> read issue #1 in this repo and tell me what's actually being asked.
  Don't fix anything yet.
> /recall empty input
```

EXPECT

An `issue_read` tool call and a two-line summary that mentions the divide-by-zero. `/recall` returns nothing the first time. Say so: “*Empty now. Watch it fill up by M9.*”

IF IT BREAKS

MCP auth failure: paste the issue body into the prompt and move on. The point is the stage, not the plumbing.

SAY

Every issue gets its own directory. This is a bare clone with sibling worktrees: one git database, several real checkouts. An agent working on #1 can't touch #2's files.

When I `cd` in, direnv loads the Rust toolchain and **links a Rust specialist agent into the project**. It isn't installed globally, because every globally installed agent costs tokens in every session.

DO

```
exit                                # leave the agent
cd ~/demo && rm -rf tally
git bclone git@github.com:<you>/tally.git
cd tally/main
git wb fix/1-empty-input            # new worktree off origin/main
git meta init && git meta link      # share .envrc across worktrees
cd ../fix/1-empty-input
# direnv: loading .envrc ...
ls -la .claude/agents/              # → rust-engineer.md symlink
tree -L 2 -a ..                     # show the layout
```

EXPECT

A `.claude/agents/rust-engineer.md` symlink appears on `cd`. Then `cd` into a non-Rust dir to show it links nothing and leaves no directory behind.

IF IT BREAKS

`direnv: .envrc is blocked → direnv allow`. Point out that this is a security feature: the same gate stops a malicious repo from running code on `cd`.

M3 · Unattended and safe

SAY

“I run the agent with permission prompts **off**. That sounds reckless, so here's why it isn't.” Walk through the layers top-down: the sandbox is the wall, the hooks are the smoke detector.

- **Sandbox:** bubblewrap, read-only system, only project and cache dirs writable, no home directory secrets.
- **Bash guard:** every shell command is parsed before it runs. Push, hard reset and curl-pipe-to-shell are denied; `sudo`, `rm` and `dd` ask.
- **Deny list:** secrets and key files can't be read even inside the sandbox.

DO

```
a
> discard all my local changes with git reset --hard
> install rustup with curl https://sh.rustup.rs | sh
> read ~/.ssh/id_ed25519
> use sudo to clean the target directory
> ok, just run cargo test
```

EXPECT

git reset --hard and the curl-pipe are **denied** by the hook with a reason, even with permission prompts off. The key read is blocked by the deny list. sudo stops and **asks** you. cargo test runs normally and shows the panic from issue #1.

Point out the split: irreversible or outward-facing actions (push, hard reset, clean) are never the agent's call. Risky-but-local ones ask.

OPTIONAL DEEP CUT, 2 MIN

Show test-fixtures/bypass-payloads.txt: s\udo, "sudo", \$(echo sudo) ... The guard is tested against 38 obfuscation tricks in a VM test on every check. This works well with security-minded audiences.

IF IT BREAKS

If the model refuses to even try (it sometimes does), say “the model refused, and if it hadn't, the hook would have” and call the hook directly:

```
hook=$(jq -r '.hooks.PreToolUse[] | select(.matcher=="Bash") | .hooks[0].command'
~/.claude/settings.json)
echo '{"tool_input":{"command":"git reset --hard"}}' | "$hook"; echo "exit=$?"
```

If \$hook comes back empty, the guard isn't enabled. See the T-1 day checklist.

M4 • Parallel

0:42 • 8 min • slide 8

SAY

Isolation plus safety means I can run several at once. The problem then is knowing **which one needs me**. The tab title tells me, and the phone buzzes if I'm away.

DO

```
# tmux window 2
cd ~/demo/tally/main && git wb feat/2-json && cd ../feat/2-json
a
> read issue #2 and brainstorm an approach with me

# tmux window 3
cd ~/demo/tally/main && git wb fix/3-unicode && cd ../fix/3-unicode
a
> read issue #3 and write a failing test that reproduces it
```

EXPECT

Window titles cycle :: (working) → ? (window 2 asks a brainstorm question) → ✓ (window 3 finishes). Point at the statusline: context used, branch, 5 h / 7 d limits. Mention lspmux: three agents, **one** rust-analyzer.

IF IT BREAKS

Glyphs not changing: the terminal may be overriding the title. Show the statusline instead and move on.

M5 · Design

0:50 · 15 min · slide 9

SAY

“The biggest time sink with agents isn't bad code. It's **good code for the wrong design**. So before any code, we get a spec, and then a separate review checks the spec against the real codebase.”

Stress the split: fixes with no downside are applied automatically, and anything with a real trade-off is **parked for me**. I only see the decisions that are mine.

DO

```
# back in window 1, fix/1-empty-input
> let's fix issue #1. brainstorm with me first.
(answer its questions live, and ask the room for one of them:
"empty input: exit 0 with zeros, or exit 1 with an error?")
> /spec-review
```

EXPECT

A file `docs/specs/YYYY-MM-DD-empty-input-design.md` with options, pros and cons. `/spec-review` announces *Round 1/3*, runs two finders (doc quality, codebase validation), verifies findings, then shows a diff of the spec plus a short list of parked human-calls.

ROOM MOMENT

Letting the audience vote on the empty-input behaviour makes them co-owners of the spec. They'll watch the rest much more closely.

IF IT BREAKS

Spec review over 4 min: move on to the break and come back to the result. If it's truly stuck, check out `demo/done` and show the finished spec.

Break

1:05 · 10 min

DO, RIGHT BEFORE THE BREAK

```
> write the implementation plan, then execute it with subagents
```

Leave the screen on the tmux window so people see `⚡` while they're away.

M6 · Plan and build

1:15 · 10 min · slides 10-11

SAY

Open the plan file. Show the dependency graph, the agent assignments and the checkboxes that ticked during the break. “If my laptop had died during the break, a new session would read this file and continue from the first empty box. The plan is the progress log.”

Each task went to a fresh subagent with only the plan and spec: the main session stays small. Tests first, then code, and every task reviewed before the next batch.

DO

```
bat docs/plans/*.md          # or open in the editor
git log --oneline origin/main..HEAD
> /checkpoint                 # show what gets saved before a context clear
```

IF IT BREAKS

The build is still running: show the plan with partial checkboxes. That actually makes the point better. Continue with M7 on `demo/done`.

SAY

“After the build, and before I look at anything, a polish pipeline runs: three review rounds, each fixing what the last one found, then an idioms pass, then a cleanup that removes dead code and AI artefacts, then comments get trimmed and the survivors rewritten to read like a person wrote them.”

“Why three rounds? Each round of fixes can introduce new issues. Returns fall off after three. It stops early if a round comes back clean.”

DO

```
git log --oneline origin/main..HEAD    # point at the review-fix commits
> /cleanup                             # if polish didn't run, show it standalone
```

M8 • Review and ship

SAY

“Now it's my turn, and it looks exactly like reviewing a colleague's branch.” The diff is always against the merge base (three dots), so work that landed on main in the meantime doesn't clutter it.

DO

```
nvim
_cvb      # whole branch vs base
_cvl      # commit by commit
_cvf      # difftastic: structural diff
_cvq      # close

a
> /commit      # if anything is left unstaged
> /pr
gh pr create --fill      # or let the agent create it

cd ../main && git wr fix/1-empty-input    # after merge
```

EXPECT

A one-line conventional commit (`fix(parser): handle empty input`) that you confirm, and a PR description built from the whole branch. `git wr` keeps the branch if it's unmerged, so show that safety too.

MENTION, DON'T DEMO

At work, GitLab MR review happens in the same editor: discussion tree, line comments, suggestions, approve, pipeline status (`_cvr _cvc _cva _cvp`). Don't show work MRs on screen.

SAY

“The decision the room voted on (empty input exits 0) is exactly what a future session would get wrong. So we store it.”

DO

```
> /remember tally treats empty input as valid: exit 0, all counts zero
# new session in window 2 (feat/2-json)
> /recall empty input
> /reflect
```

EXPECT

The fact comes back in a different worktree, in a different session. Go back to slide 16: four points where you step in, and between them “you're on another issue in another tab.”

04 Hands-on lab

30 minutes. Attendees work in pairs on their own laptops with whatever agent they already use: Claude Code, Codex, Cursor, anything. The goal is to feel the workflow, not to install this setup.

Lab brief (put it on screen)

1. **Fork** tally and pick issue #2 or #3.
2. **Isolate:** `git worktree add -b <branch> ../<branch> origin/main`. Work only there.
3. **Design:** ask the agent to write a spec to `docs/specs/`, then ask it (in a fresh session, if you can) to critique that spec against the code. Make one decision yourself.
4. **Plan:** have it write a checkbox plan to `docs/plans/` and tick boxes as it goes.
5. **Crash test:** halfway through, quit the agent. Start a new session with only “continue the plan in `docs/plans/`”. Does it resume?
6. **Review:** `git diff origin/main...HEAD`. Would you merge it from a colleague?

Facilitator's role during the lab

- Walk the room. The two most common blockers are worktree paths (“fatal: already exists”) and agents that skip the spec and start coding. For the second, tell them to say “stop, spec first” explicitly.
- Pick one pair who hit an interesting problem and ask if they'll show it in the debrief.
- At 20 min, give a 10-minute warning. At 28 min, call time.

Debrief questions (5 min)

- Did the spec review catch anything you'd have missed?
- Did the crash test resume cleanly? What did the plan need to make that work?
- Which of the four intervention points felt most valuable?

Stretch goals for fast pairs

- Run both issues at once in two worktrees and two agent sessions.
- Add the guard hook from Chapter 5 and try to get `sudo` past it.

05 Take-home kit

Every idea from the talk in its smallest form. No Nix, no custom tooling. Share this chapter as a link after the session.

5.1 Worktree per issue

```
git config --global alias.wb '!f() { git fetch origin && git worktree add -b "$1" "../$1"
origin/main; }; f'
git config --global alias.wr '!f() { git worktree remove "../$1" && git branch -d "$1"; };
f'
git config --global alias.wl 'worktree list'

git wb fix/42      # → ../fix/42, a fresh checkout on a new branch
git wr fix/42      # remove it; keeps the branch if unmerged
```

5.2 Bash guard hook (Claude Code)

~/claude/settings.json:

```
{
  "hooks": {
    "PreToolUse": [
      { "matcher": "Bash",
        "hooks": [{ "type": "command", "command": "~/claude/hooks/guard.sh" }] }
    ]
  }
}
```

~/claude/hooks/guard.sh (chmod +x, needs jq):

```
#!/usr/bin/env bash
cmd=$(jq -r '.tool_input.command // empty')
[ -z "$cmd" ] && exit 0
# strip quotes and backslashes so "s\udo" and 'sudo' don't slip through
norm=$(printf '%s' "$cmd" | tr -d "\\\\"'"')
if grep -Eq '^(^|;|&|` )sudo([ ]);&|)|$)|rm +-[a-z]*r[a-z]*f|curl[^|]*\| *(ba|z)?sh'
<<<"$norm"; then
  echo "blocked by guard hook: $cmd" >&2
  exit 2      # exit 2 = deny; stderr is shown to the agent
fi
exit 0
```

BE HONEST ABOUT IT

A regex guard stops accidents, not someone determined to get past it. If you turn permission prompts off, also run the agent in a sandbox (a container, a VM, or Claude Code's built-in sandbox). The hook is the smoke detector; the sandbox is the wall.

5.3 Scoped agents

Put specialist agents in the **project's** `.claude/agents/` rather than `~/.claude/agents/`. Everything global costs prompt tokens in every session. Same for MCP servers: enable per project, and disable tools you never call.

5.4 Spec before code

```
Before writing any code, write a design spec to docs/specs/<date>-<topic>.md:
problem, options (with pros and cons for each), chosen approach, test plan.
Stop and wait for my approval.
```

Then, in a **fresh** session:

```
Review docs/specs/<file>. For every claim it makes about the codebase, verify it
by reading the code and cite file:line. List logic gaps and false assumptions.
Fix what has no trade-off; list the rest for me to decide.
```

5.5 Plan as progress log

```
Write a plan to docs/plans/<topic>.md as a checkbox list of small tasks,
each with its test. Mark a box [x] only after its test passes, and commit
after each task. If you're restarted, resume from the first unchecked box.
```

5.6 Know when to look

```
{
  "hooks": {
    "Notification": [{ "matcher": "", "hooks": [{ "type": "command",
      "command": "notify-send 'Agent' 'needs you'" }] }],
    "Stop": [{ "matcher": "", "hooks": [{ "type": "command",
      "command": "notify-send 'Agent' 'done'" }] }]
  }
}
```

On macOS use `osascript -e 'display notification "needs you" with title "Agent"'`. For a phone: `curl -d "needs you" ntfy.sh/<a-long-random-topic>`.

5.7 Clean before review

```
Review git diff origin/main...HEAD. Remove: comments that restate the code,
dead code, unused imports, abstractions used once, "AI narration". Keep only
comments that explain why. Match the surrounding code's style. Commit.
```

06 Facilitation

Pacing

- **Say the stage name out loud** at the start of every module (“We’re at *Isolate*”). The lifecycle is what people will remember.
- **Talk while the agent works.** Plan what you’ll say during each wait. Silence while a spinner turns loses the room.
- **Cut from the middle, not the end.** If you’re 10 min behind, drop M4 (parallel) to a slide-only mention and shorten M7. Never cut the hands-on lab or M8.

Hard questions, and honest answers

QUESTION	ANSWER
“Isn’t skipping permissions dangerous?”	Without a sandbox, yes. The claim isn’t that the hook is enough. The sandbox limits what’s reachable, and the hook catches the obvious mistakes early with a readable reason. Show the layers slide again.
“What does this cost?”	Be concrete about your own plan and usage. The statusline shows rate limits for exactly this reason. Spec review and three polish rounds do spend tokens; the trade is fewer wasted build cycles on a wrong design.
“Does the code quality hold up?”	It holds up as well as your review does. The polish pipeline raises the floor, but the human diff review is not optional, and it’s one of the four points where you step in.
“Will this replace developers?”	Look at where the time went today: deciding the design, answering trade-offs, reviewing. The typing moved to the agent; the judgement didn’t.
“Why worktrees, not branches?”	A branch switch changes files under a running agent. A worktree is a separate directory, so two agents never share files.
“Does it work with Codex / Cursor / ...?”	The ideas do. In this setup the same shared commands, rules and agents feed both Claude Code and Codex from one source.
“What about private code and data?”	Memory is a local vector DB. For the model provider’s data handling, point to your org’s policy, and don’t improvise legal answers.

When a demo goes wrong

Say what happened in one sentence, then switch to the fallback. Agent failures are useful teaching moments. If the agent does something silly, ask the room: “*Which stage should have caught that?*” Usually it’s the spec review or the polish, which reinforces the lifecycle.

07 Troubleshooting

SYMPTOM	FIX
<code>direnv: .envrc is blocked</code>	<code>direnv allow</code> in that worktree.
No <code>.claude/agents/</code> after <code>cd</code>	<code>direnv</code> didn't load (check the <code>direnv:</code> line), or the project doesn't match a detector. Force it with <code>use claude_agents rust-engineer</code> in <code>.envrc</code> .
<code>git wb</code> : “already exists”	A leftover worktree from a rehearsal. Run <code>git wl</code> , then <code>git wr <branch></code> or <code>git worktree prune</code> .
<code>git fetch</code> in a bare clone doesn't update <code>origin/*</code>	The clone predates <code>git bclone</code> 's refspec fix: <code>git config remote.origin.fetch '+refs/heads/*:refs/remotes/origin/*'</code> .
MCP server “failed to connect”	Run <code>/mcp</code> in the agent to see status. Usually expired auth: <code>gh auth status</code> . Fallback: paste the issue text.
<code>/recall</code> errors	The vector DB isn't running. Start <code>qdrant</code> , or skip M9's live part and describe it.
Tab glyphs never change	The terminal is overriding titles, or you're outside <code>tmux</code> . Show the statusline instead.
Blocked commands just run	The bash guard is off by default. Set <code>custom.claude.bashGuard.enable = true</code> for the host and <code>just switch</code> .
Agent refuses to try a blocked command	Pipe a payload into the hook directly (see M3 fallback).
Rate limit hit mid-demo	Switch to recordings for the remaining modules. Check the statusline during setup next time.
Build still running after the break	Show the partial plan checkboxes, then check out <code>demo/done</code> .
Projector too small for <code>nvim diff</code>	Use <code>git diff origin/main... HEAD</code> in the terminal with a big font, or <code>diffit</code> .

08 Appendix

8.1 Cheat sheet

STAGE	COMMAND	WHAT IT DOES
Pick	<code>/recall <query></code>	Search project memory (filters: <code>type:</code> <code>entity:</code> <code>date:7d</code>)
Isolate	<code>git bclone <url></code>	Bare clone + worktree for the default branch
	<code>git wb <branch></code> / <code>git wr <branch></code> / <code>git wl</code>	New / remove / list worktrees
	<code>git meta init link status</code>	Share <code>.envrc</code> , archive specs and plans
	<code>a</code> / <code>ar</code>	Sandboxed Claude / resume a session
	<code>o</code> / <code>or</code> / <code>ox</code>	Sandboxed Codex / resume / exec
Design	“brainstorm with me”	Questions, options, spec in <code>docs/specs/</code>
	<code>/spec-review</code>	Up to 3 rounds: find, verify, autofix, park trade-offs
Plan/Build	“write the plan, execute with subagents”	Task graph, agent assignment, checkboxes
	<code>/checkpoint</code>	Save session state before a context clear
Polish	<code>/cleanup</code>	Simplify, remove AI artefacts, question the decisions made
Review	<code>_cvb</code> <code>_cvl</code> <code>_cvf</code> <code>_cvo</code> <code>_cvw</code>	Branch diff, commit walk, difftastic, pick MR, open in web
	<code>/code-review</code>	Review agent over the branch
Ship	<code>/commit</code> · <code>/pr</code>	One-line conventional commit · PR from the whole branch
Remember	<code>/remember</code> · <code>/docs</code> · <code>/reflect</code> · <code>/forget</code>	Store · update docs · audit · delete

8.2 Presenter file map

For when someone asks “how is that done?”. Paths are relative to the dotfiles repo.

FEATURE	WHERE
Worktree tooling	<code>scripts/git-bare-clone.sh</code> , <code>git-worktree-new.sh</code> , <code>git-worktree-remove.sh</code> , <code>git-meta.sh</code> ; aliases in <code>home/common/git.nix</code>
Scoped agents and rules	<code>claude/direnv-agents.sh</code> , <code>claude/direnv-rules.sh</code> ; sources in <code>llm/shared/{agents,rules}/</code>
Sandbox	<code>home/linux/sandboxed.nix</code> , <code>home/darwin/sandboxed.nix</code>
Guard hooks, deny list	<code>claude/security/default.nix</code> , <code>claude/security/scripts/check-bash-command.sh</code>
Bypass tests	<code>claude/security/scripts/test-fixtures/</code> , <code>tests/claude-security.nix</code>
Tab state, statusline	<code>claude/tab-state.sh</code> , <code>claude/statusline.sh</code>
Shared LSP	<code>home/common/lspmux.nix</code>
MCP servers	<code>home/common/mcp.nix</code>
Slash commands	<code>llm/shared/commands/</code>
Spec review, polish	<code>llm/shared/skills/spec-review.md</code> , <code>post-implementation-polish.md</code>
Brainstorm/plan/build flow	<code>superpowers plugin</code> + <code>patches/superpowers-customizations.patch</code>
Editor review	<code>config/nvim/lua/plugins/review.lua</code>
Memory	ferrex MCP in <code>home/common/mcp.nix</code> , qdrant in <code>home/common/qdrant.nix</code>

8.3 Feedback form (send within 24 h)

1. Which stage will you try first on your own work?
2. What was the most confusing part?
3. Was the pace: too slow / right / too fast?
4. What would you cut? What would you add?
5. Did anything from the take-home kit not work on your machine?